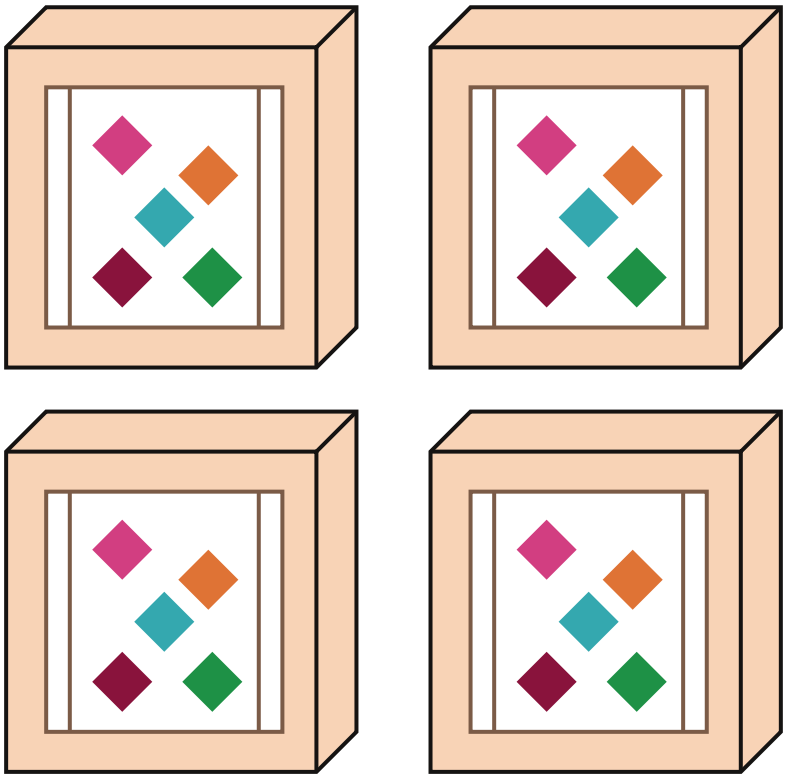

Gossipping Unikernels

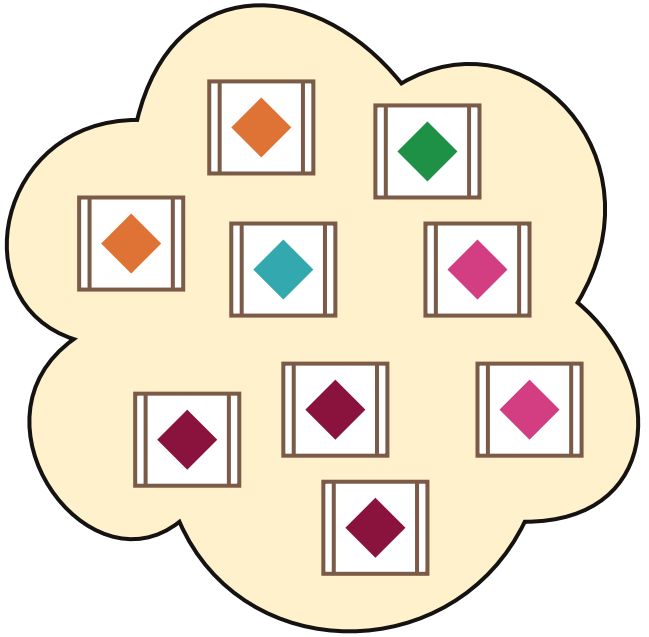
Andreas Garnæs

andreas

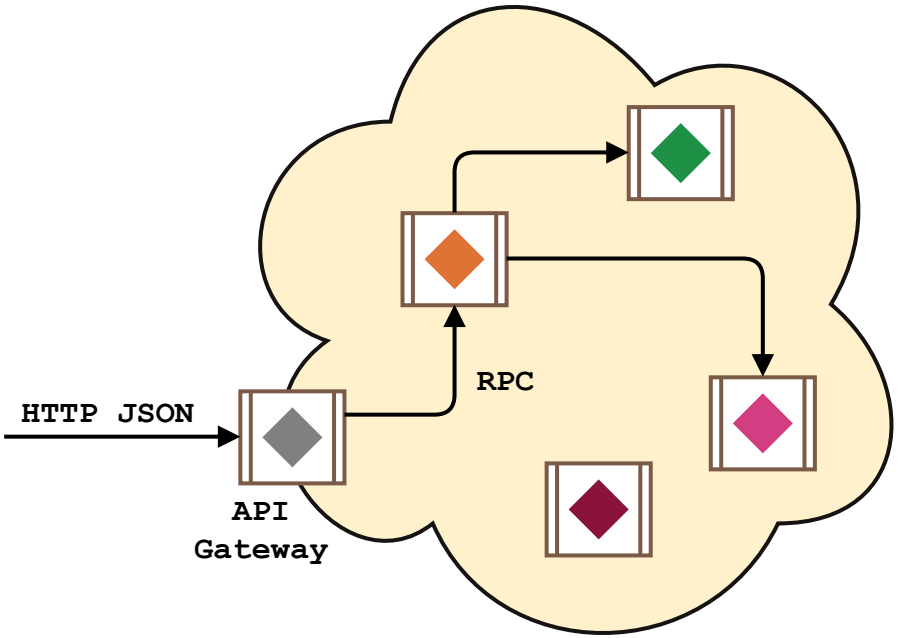
@cuvius

Microservices



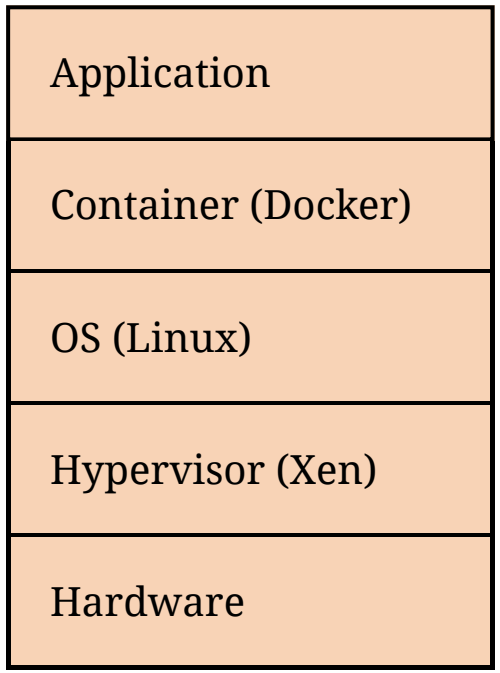


Example

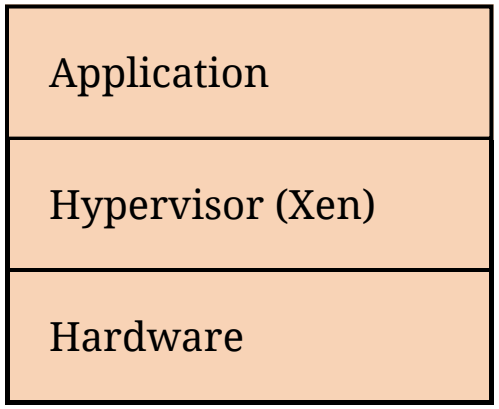


Stack

Traditional

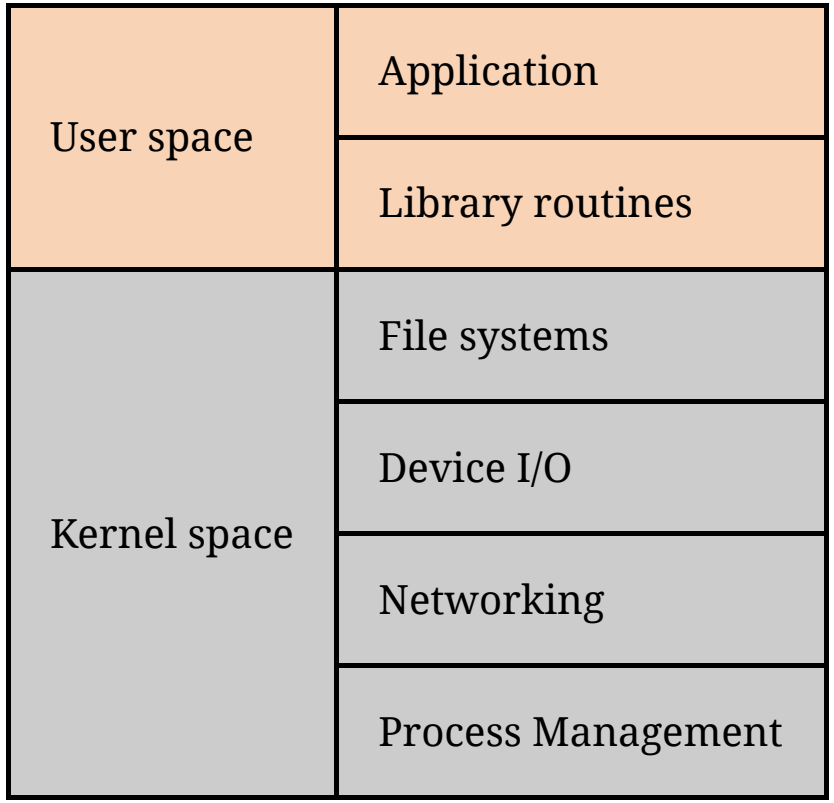


Unikernel

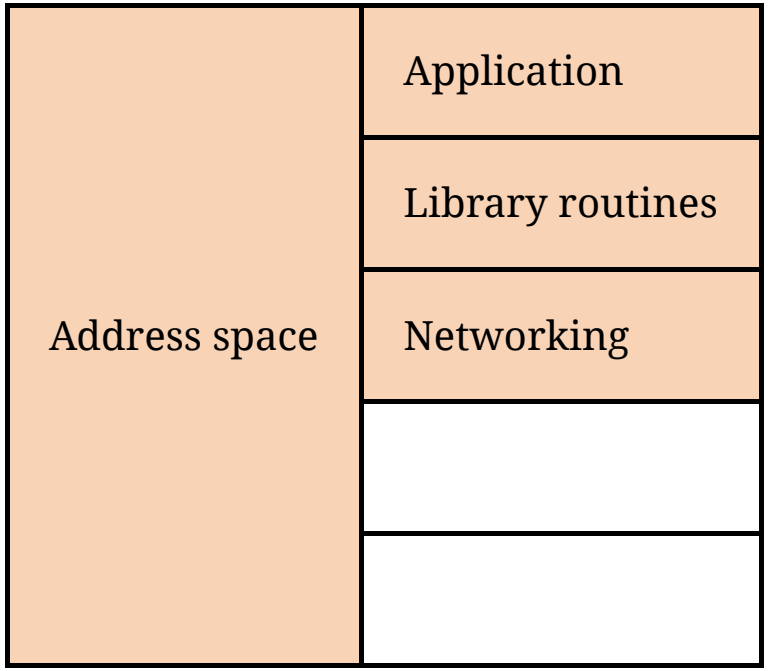


Address Space

Traditional

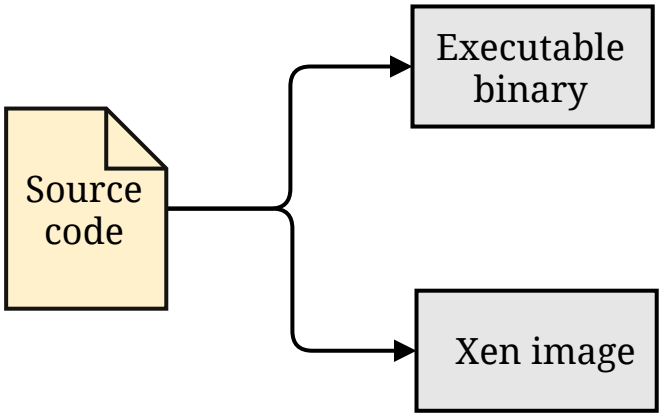


Unikernel



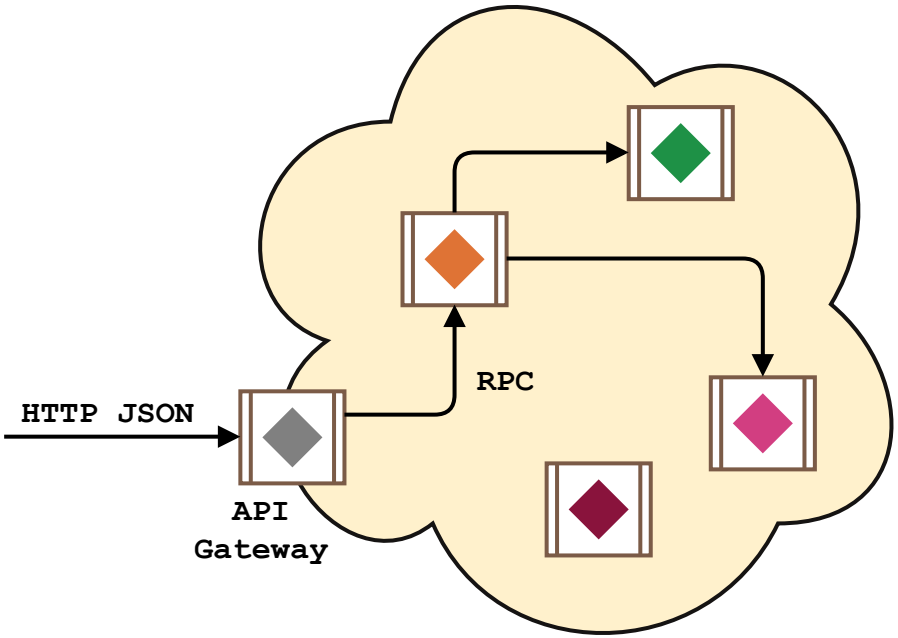
Mirage

```
module Main (Console : CONSOLE) = struct
  let start console =
    Console.log_s console "Hello, world."
end
```




```
> mirage configure --unix
> mirage build
> ./main.native
Hello, world.
```

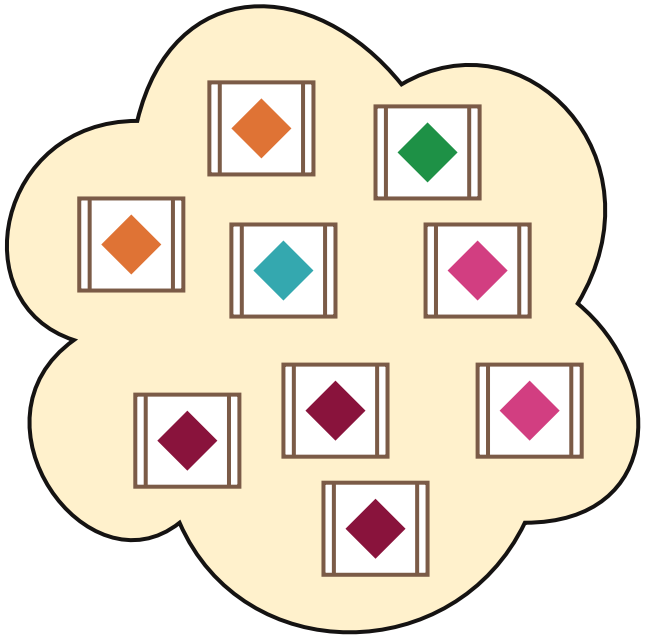
Requirements



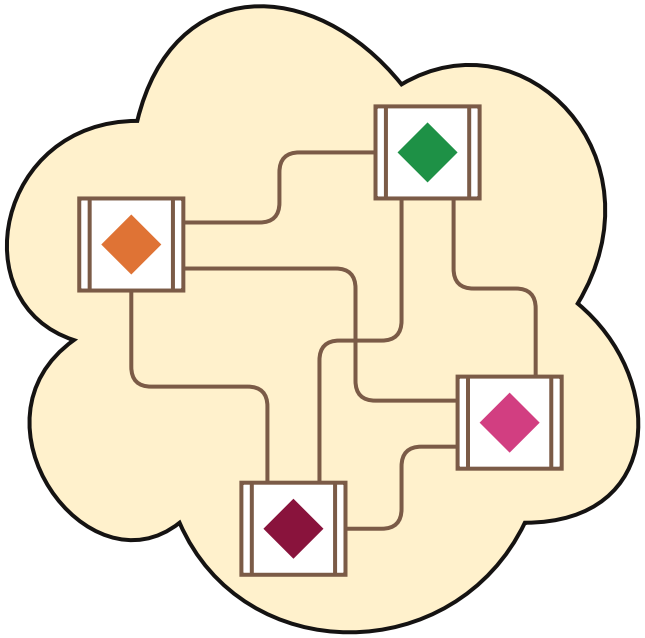
- RPC
- Service Discovery
- Deployment
- Monitoring


WELCOME TO
RESPECT PRIVACY

Service Discovery



Service Discovery

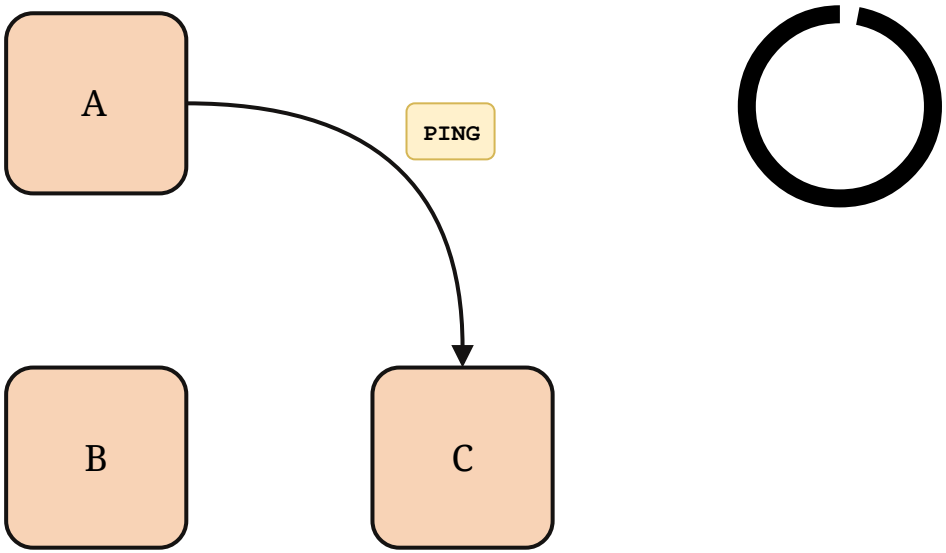


- **Completeness**: All non-faulty members eventually detect a faulty node
- **Speed:** Time to detect failures
- **Network load**: Messages on the network
- **Accuracy**: False positives

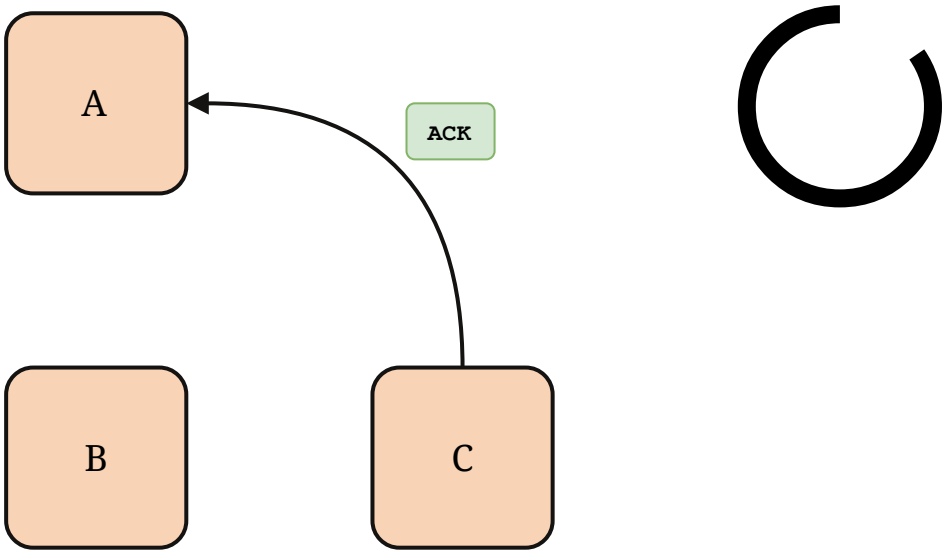
SWIM

- Peer-to-peer protocol over UDP
- Randomized heartbeating with gossipping on top
- Configurable ping interval
- 3 types of messages
  - `PING`
  - `ACK`
  - `PING-REQ`
- Trade-off: Slightly lower speed for much lower message load

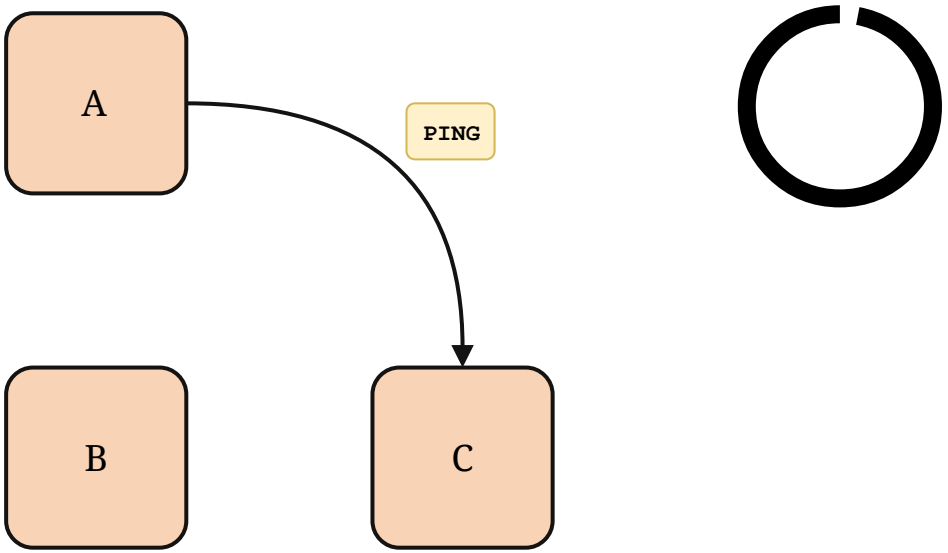
SWIM (1)



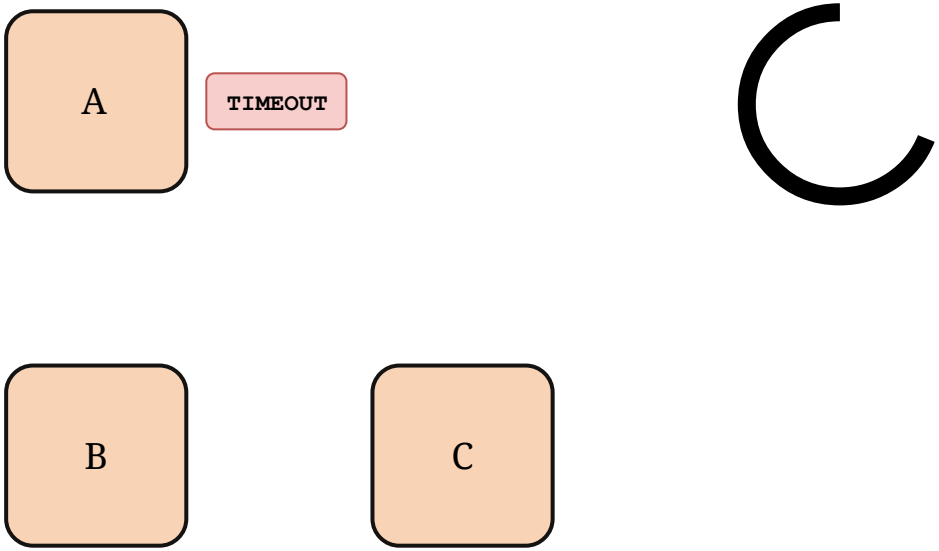
SWIM (1)



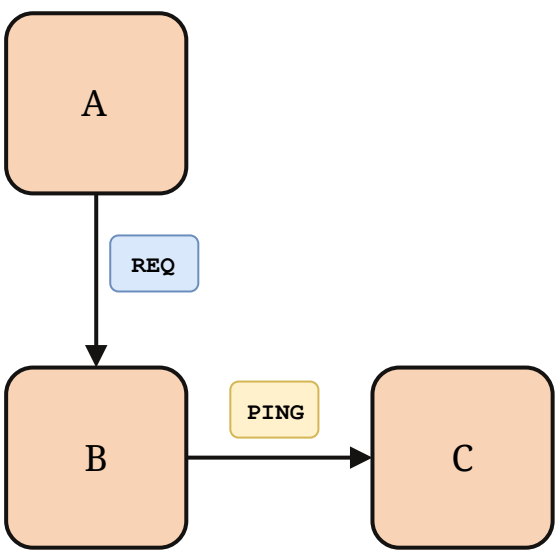
SWIM (2)



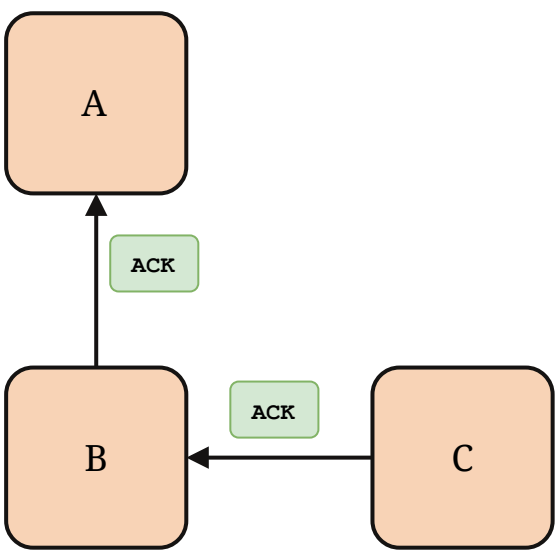
SWIM (2)



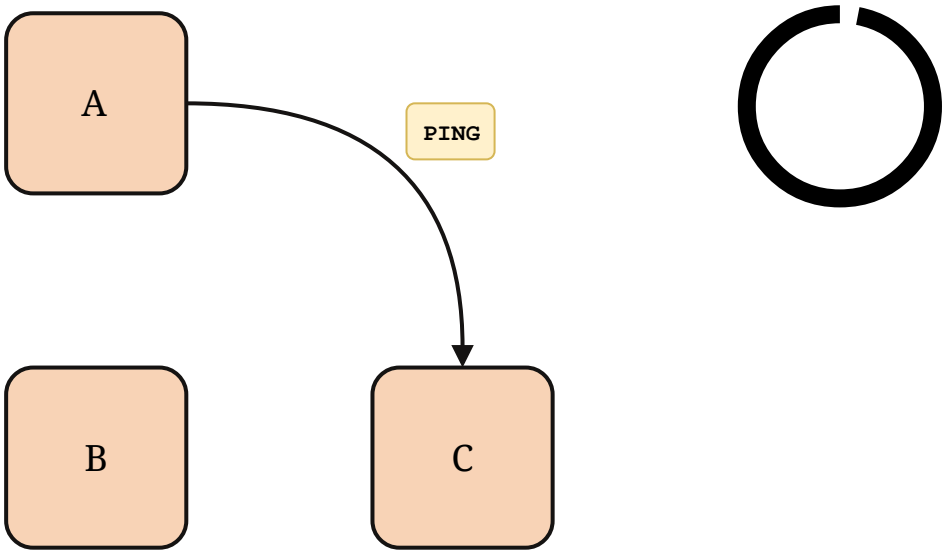
SWIM (2)



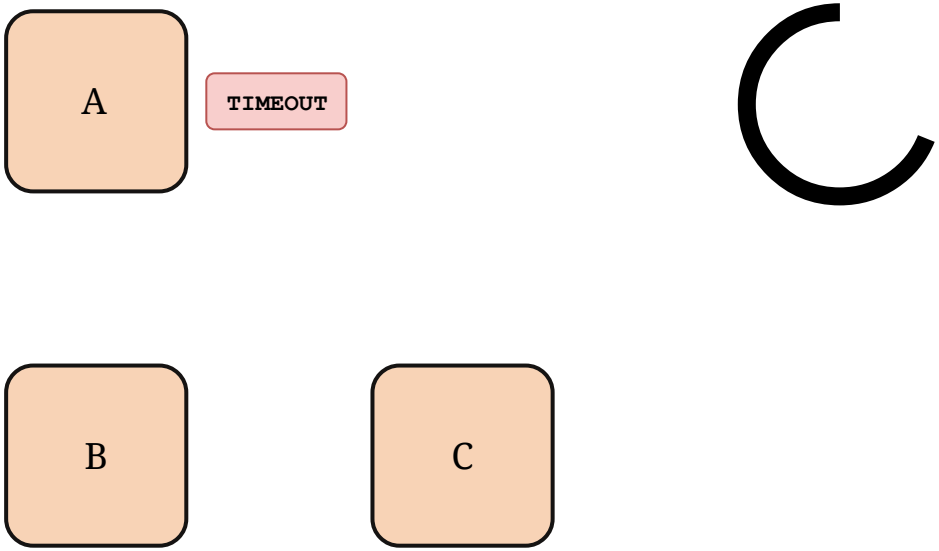
SWIM (2)



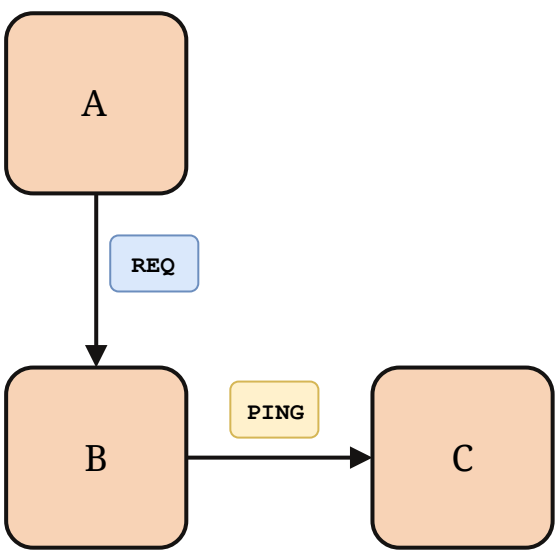
SWIM (3)



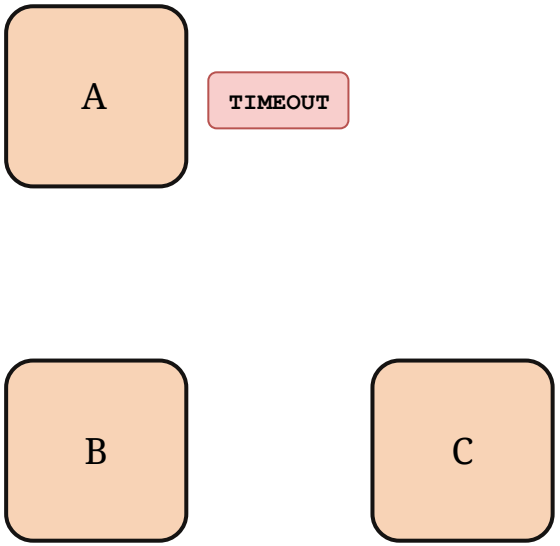
SWIM (3)



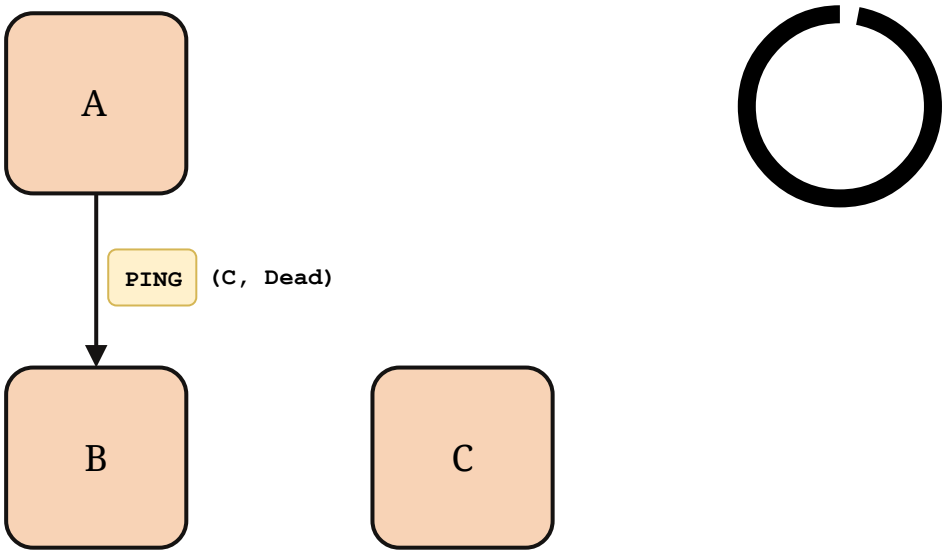
SWIM (3)



SWIM (3)



SWIM (3)



Extensions

- Increase accuracy
- Full sync on join
- Node metadata

Messages

```
type addr = ip * port

type message_type =
  | Ping
  | Ack
  | PingReq of addr

type state = Dead | Alive

type message = {
  seq_no       : int;
  message_type : message_type;
  gossip       : (addr * state) list;
}
```

Purity

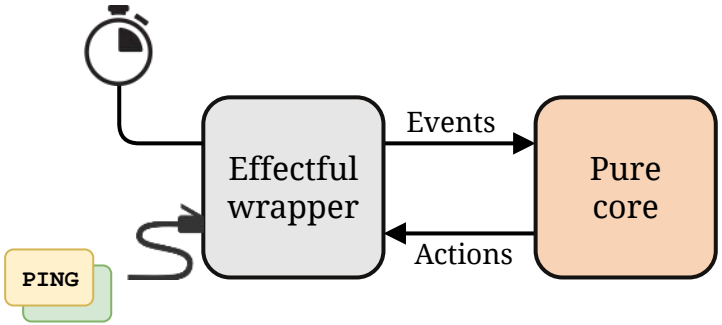
```
val handle_msg : state -> message -> unit
```

```
val handle_msg2 : state -> message -> state
```

```
type action =
  | SendMessage of addr * message
val handle_msg3 : state -> message -> state * action list
```

```
type timer (* Opaque type for timeouts *)

type action =
  | SendMessage of addr * message
  | Timer       of duration * timer

type event =
  | ReceiveMessage of addr * message
  | Timeout        of timer

val handle_event : state -> event -> state * action list
```

Effectful Wrapper



Sync	Async	Mock	Mirage
Pure core			

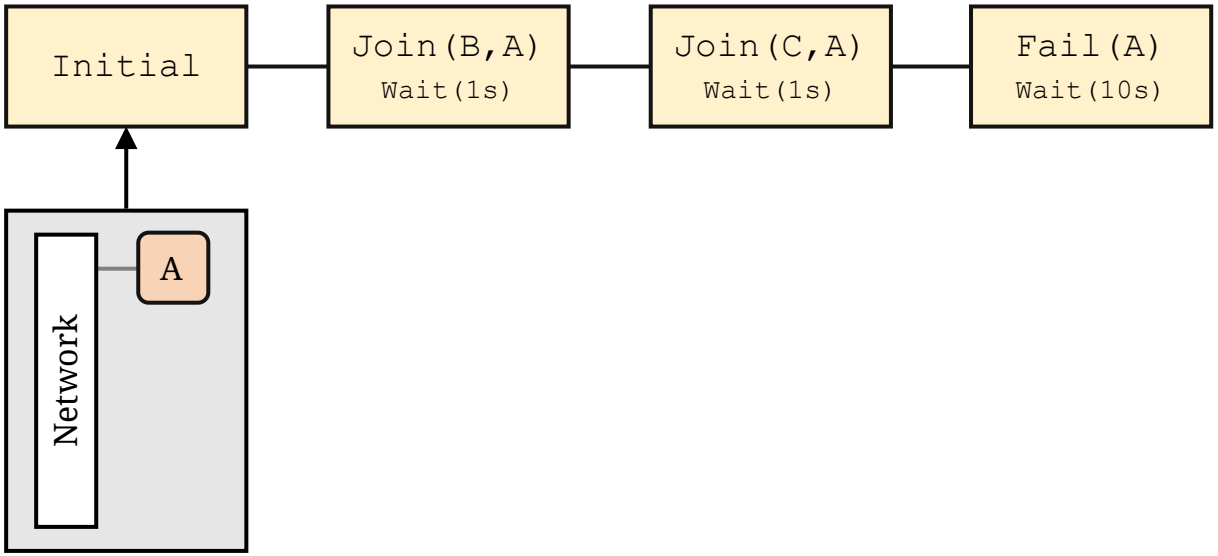
SWIM Core

```
module type SwimCore = sig
  type t     (* State of a single SWIM node *)
  type timer (* Opaque type for timeouts    *)

  type addr = ip * port

  type event =
    | ReceiveMessage of addr * message
    | Timeout        of timer

  type action =
    | SendMessage of addr * message
    | Timer       of duration * timer

  val create  : config -> t
  val members : t -> member list

  val failure_detection : t -> t * action list
  val handle_event      : t -> event -> t * action list
end
```

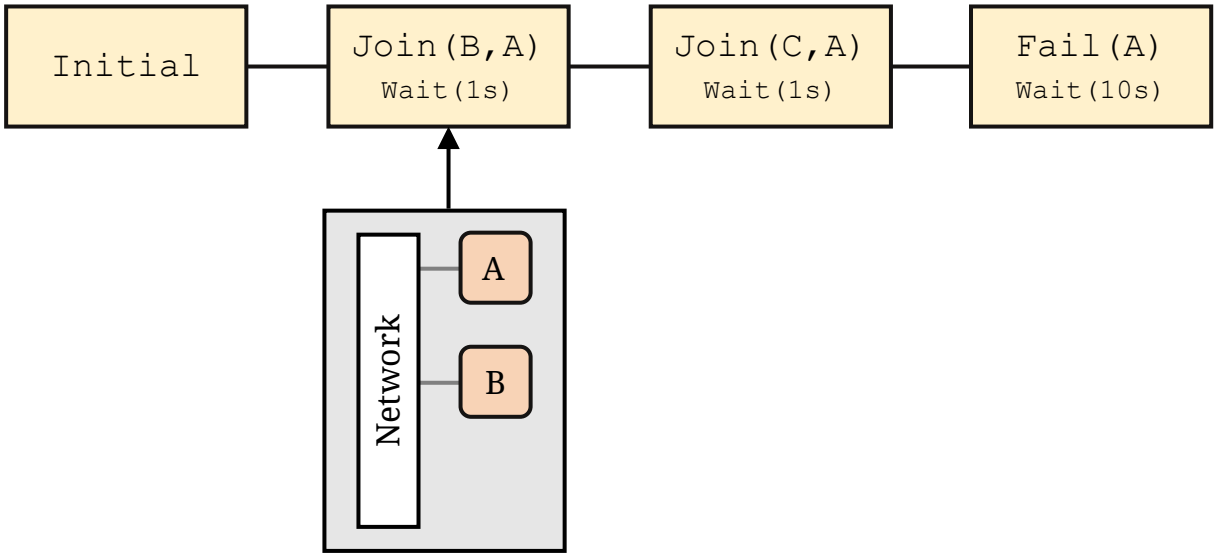
SWIM Mirage

```
module type SwimMirage = sig
  type t

  val create  : config -> t
  val start   : t -> unit
  val stop    : t -> unit
  val join    : t -> addr -> unit
  val members : t -> member list
end
```

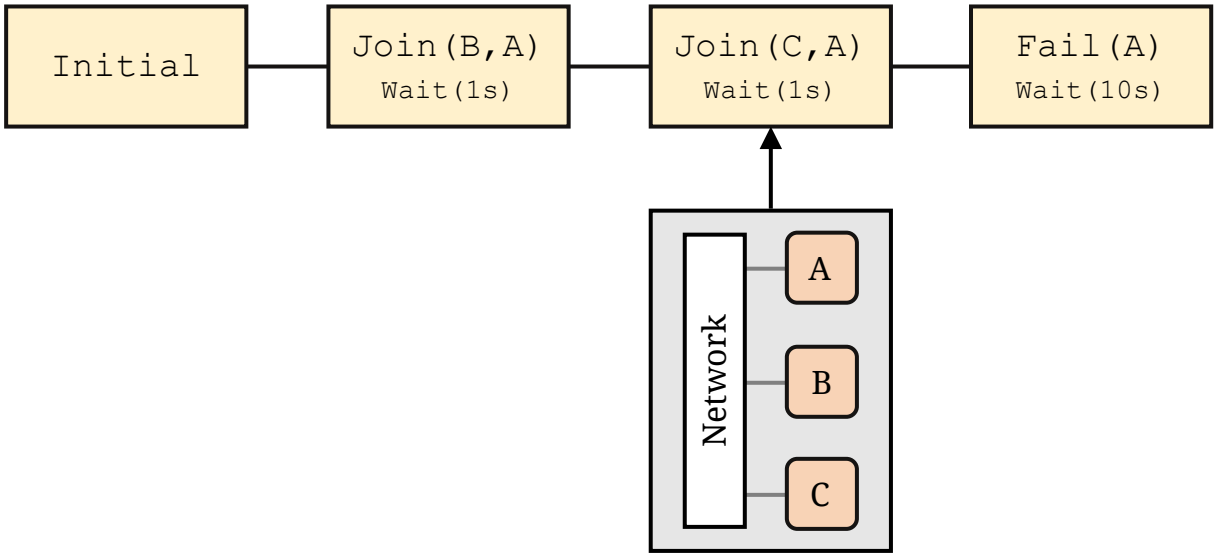
Testing


Initial
Join(B,A)
Wait(1s)
Join(C,A)
Wait(1s)
Fail(A)
Wait(10s)
Network
A

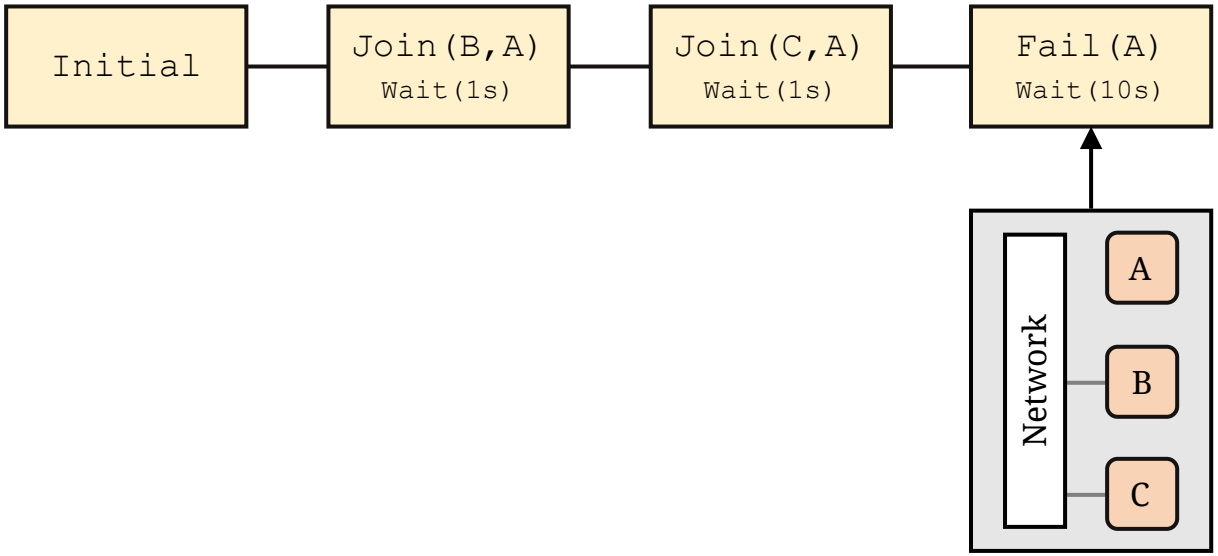
Testing


Initial
Join(B,A)
Wait(1s)
Join(C,A)
Wait(1s)
Fail(A)
Wait(10s)
Network
A
B

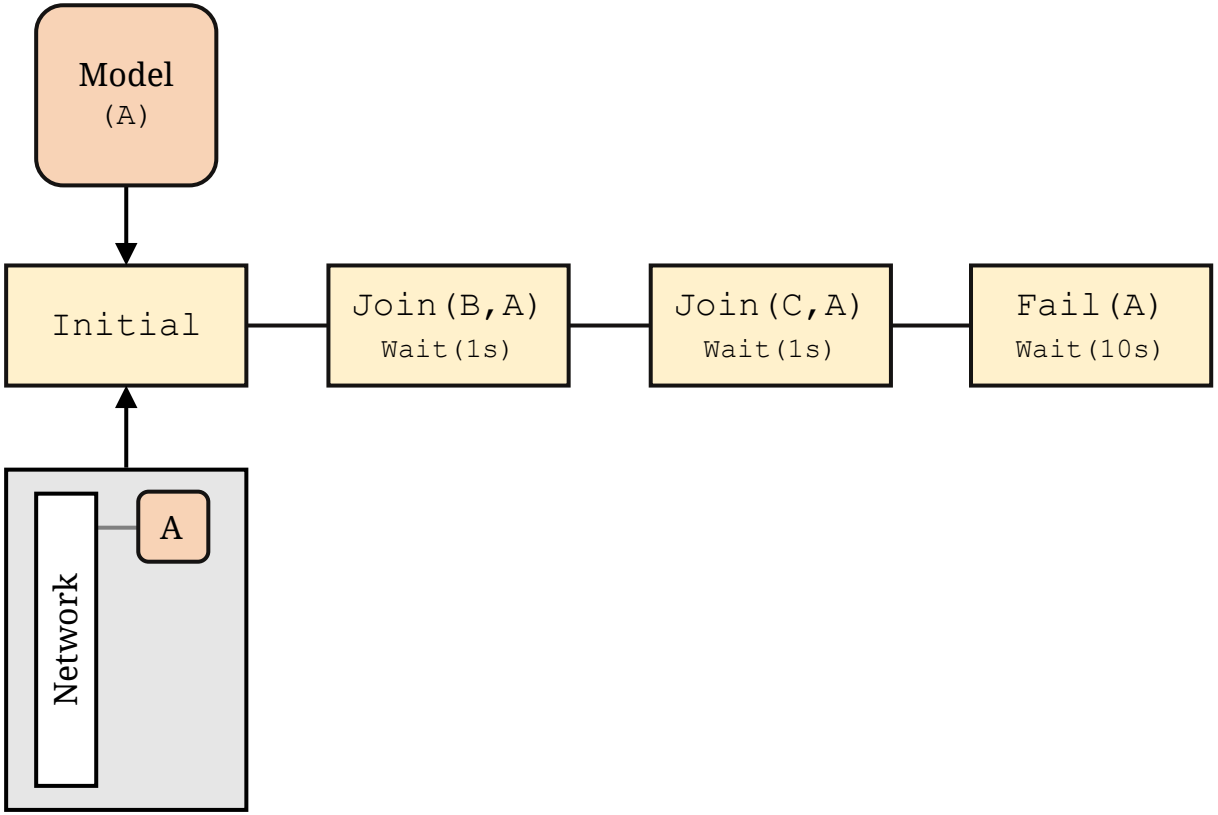
Testing


Initial
Join(B,A)
Wait(1s)
Join(C,A)
Wait(1s)
Fail(A)
Wait(10s)
Network
A
B
C

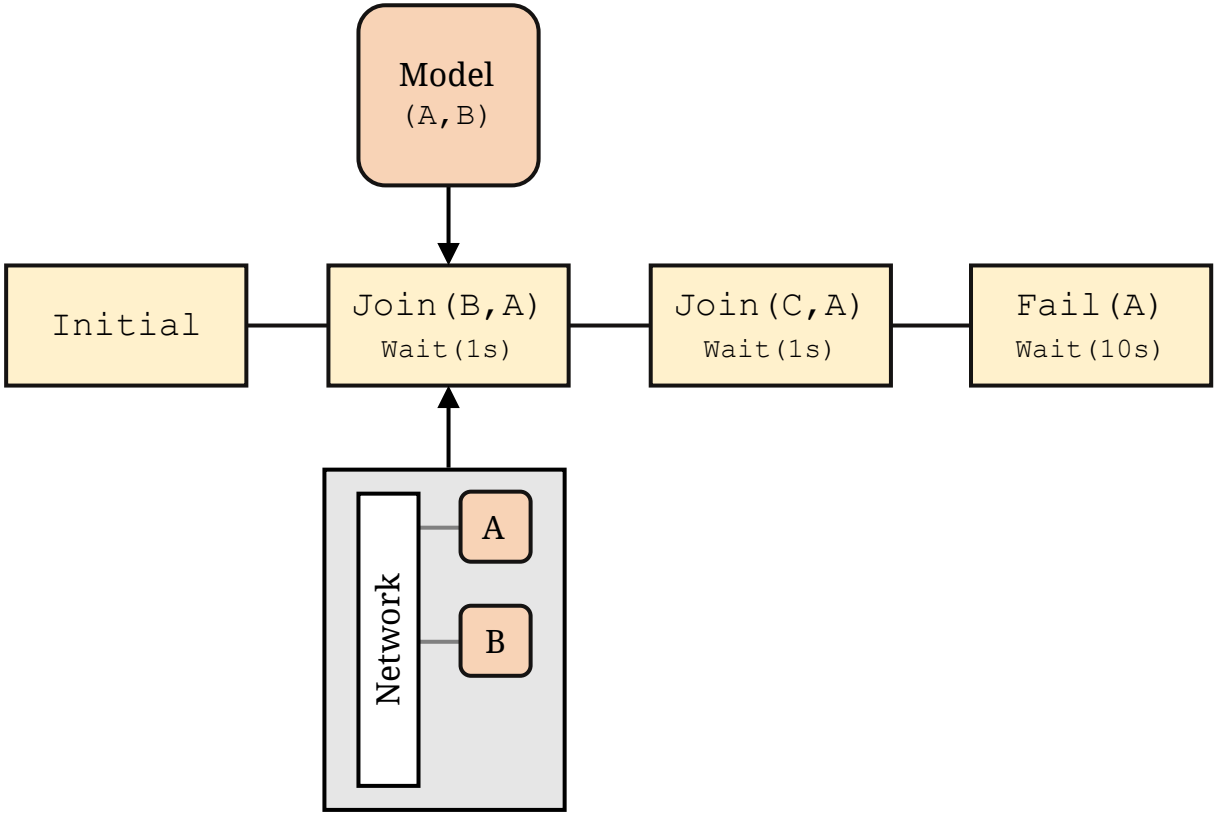
Testing


Initial
Join(B,A)
Wait(1s)
Join(C,A)
Wait(1s)
Fail(A)
Wait(10s)
Network
A
B
C

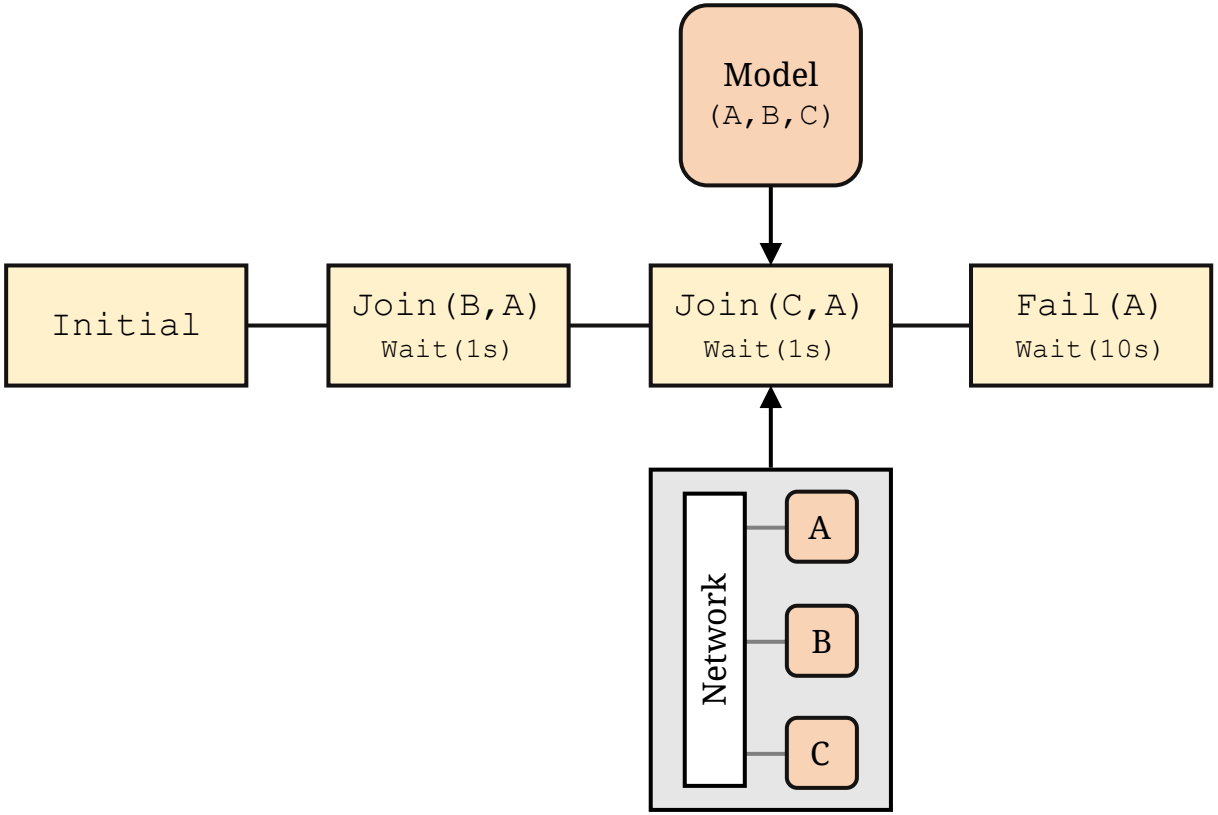
Property Based Testing



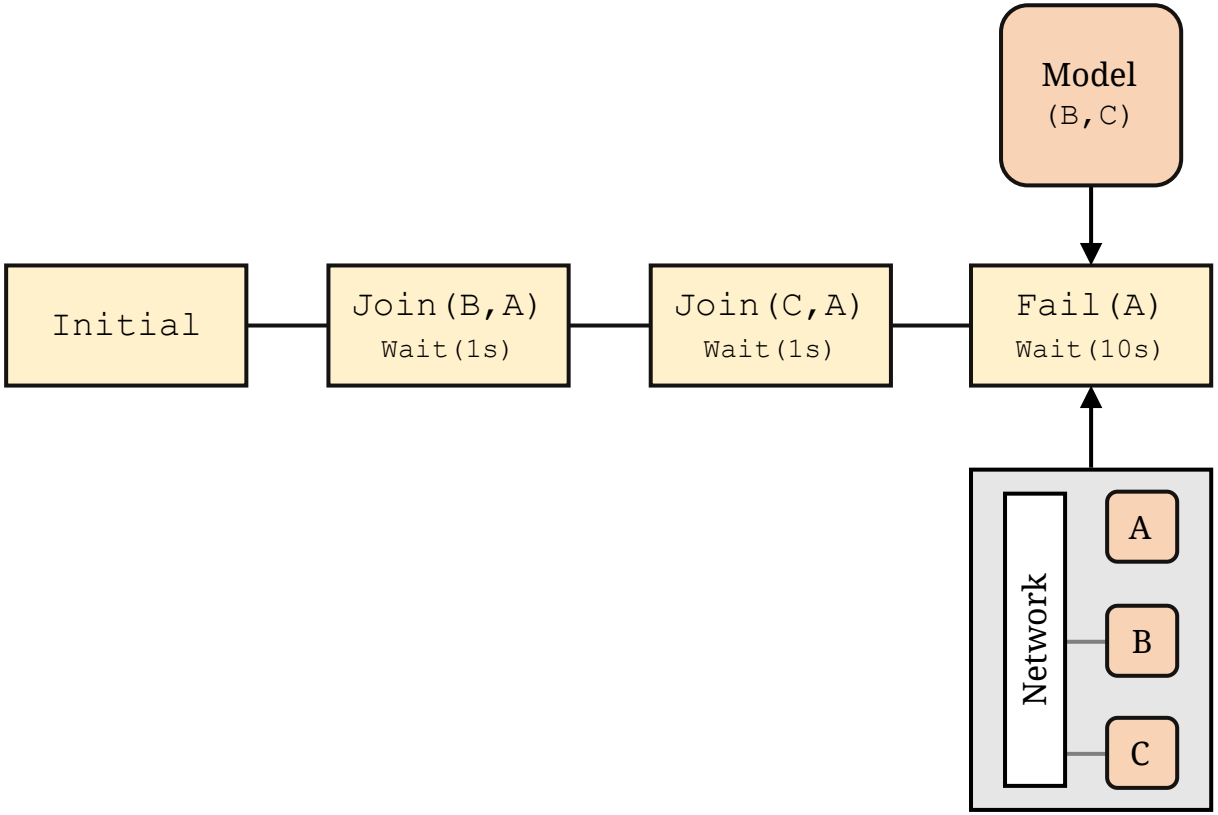
Property Based Testing



Property Based Testing



Property Based Testing



Property Based Testing

```
type event =
  | Join of addr * addr
  | Fail of addr
  | Wait of duration

type scenario = event list

type state = {
  model : addr Set.t;
  nodes : SwimCore.t list;
}

val step     : state -> event -> state
val execute  : state -> scenario -> state
val validate : state -> unit (* raises on failure *)
```

Recap

- Microservices
- Unikernels
- Service Discovery
- Next up
  - Deployment
  - Monitoring

Thanks!

Image attributions:

- Communication vector designed by Freepik
- Mirage hackathon photo by Enguerrand Decorne